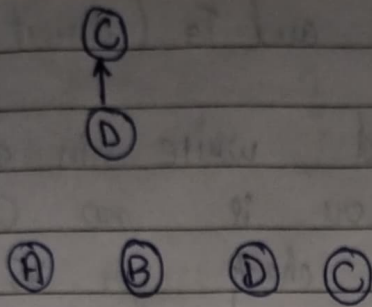
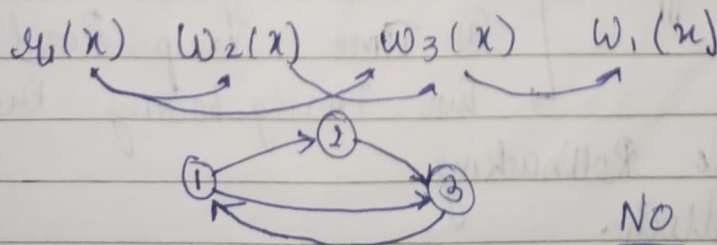


View Serializability : (VS)

→ There are some schedule which are serializable but does not satisfy CS

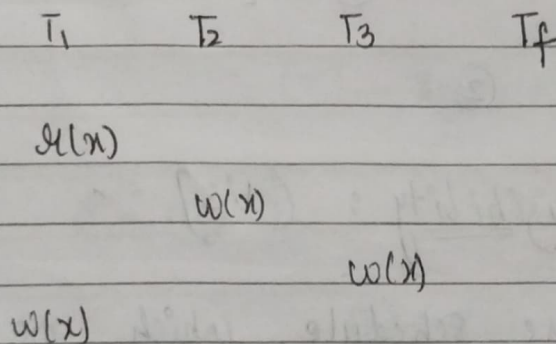
T_1	T_2	T_3
$R_1(x)$		
	$W_2(x)$	
		$W_3(x)$
$W_1(x)$		



No CS

Blind Write in T_2 and T_3 (direct write)

- When there is blind write in a schedule and there is no CS then we have to check for VS.
- If no blind write then every CS is VS.
- All R, w are checked serially in CS.
- In VS we have to check only read operation serially.



$R(x) \rightarrow$ If we add this read op in last then it becomes VS.

↳ Practically we have concurrency control Protocol to ensure serializability

Two types:

- ① 2PC
- ② Time - Stamp Based.
- ↳ Two Phasing locking Protocol.

Basic 29h

- ↳ Cascade Rollbacking
- ↳ Deadlock.

2Pb

- Shared lock
 - Exclusive lock
- Two type of locks → only for read
→ for read and write.

Phases

- growing → when data item acquire lock.
- shrinking → when data item starts unlocking

In this transaction have to acquire lock to access data.

	req lock	S	X
S	Yes	Yes	No
X	Yes	No	No

T_1
X(A)
 { R(A)
 W(A)
 X(B) } growing phase.

Unlock(A)
 R(B)
 W(B)
 Unlock(B)
 commit. } shrinking phase.

job tak 8 pe lock-acquire nahi karenge
 jab tak 7 ko lock nahi unlock nahi kar
 sakte.

Deadlock

T_1	T_2
$X(A)$	
	$X(B)$
$X(B)$	
	$X(A)$

Here T_1 waiting for unlock of B
and T_2 waiting for unlock of A
then it is deadlock condition.

Cascade Rollbacking

T_1	T_2
$X(A)$	
$RL(A)$	
$W(B)$	
$X(B)$	
$Unlock(A)$	

 ~~$X(A)$~~
 $R(B)$
 ~~$W(A)$~~ WB
 ~~$Unlock(A)$~~
 $R(B)$
 $y(B < 200)$

Rollback

Agar ek transaction fail ho jaye aur uske data par dusri transaction depend ho to sabko rollback karna padta hai.

Timestamp Based Transaction :

T_i will perform work A only when

Write Time-stamp(A) $\leq$ time stamp(T_i).

otherwise

T_i will rollback itself.

T_i will perform work A only when

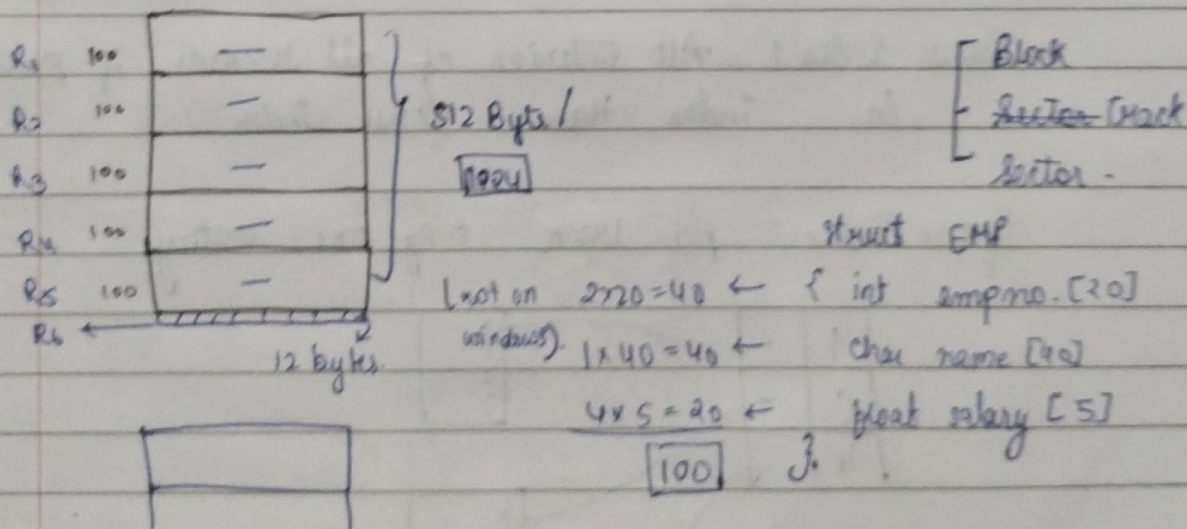
Read-Time-stamp(A) $\leq$ time stamp(T_i)

and Write Time-stamp(A) $\leq$ time-stamp(T_i).

Otherwise T_i will rollback.

Flashing and Indexing :

data is either sorted or unsorted.



At runtime we want to read as much

① $B_1 \rightarrow 12B$ } spanned file organisation of plane }
 $B_2 \rightarrow 88B$ } earlier.

② $B_2 \rightarrow 100$ } unspanned file organisation - { fast }

- FAT-32
- NTFS
- Ext.
- B+ trees.

→ Primary index

Date: / / Page no:

KF	RP	
1		1
2		2
3		3
4		4
5		5

KF: Key Field

RP: Record Pointer

Index is sorted on key field.

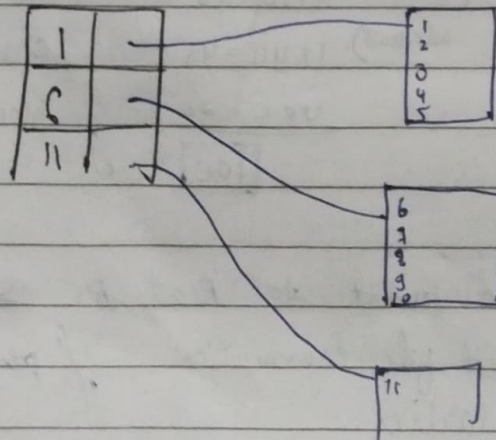
Types: → sparse index
of index: → dense index

100 Blocks → 500 Records

Total no. of records $\Rightarrow \frac{500}{5} = 100$ blocks
record in a block

Dense Index: All Entries of all records if present in index then dense index.

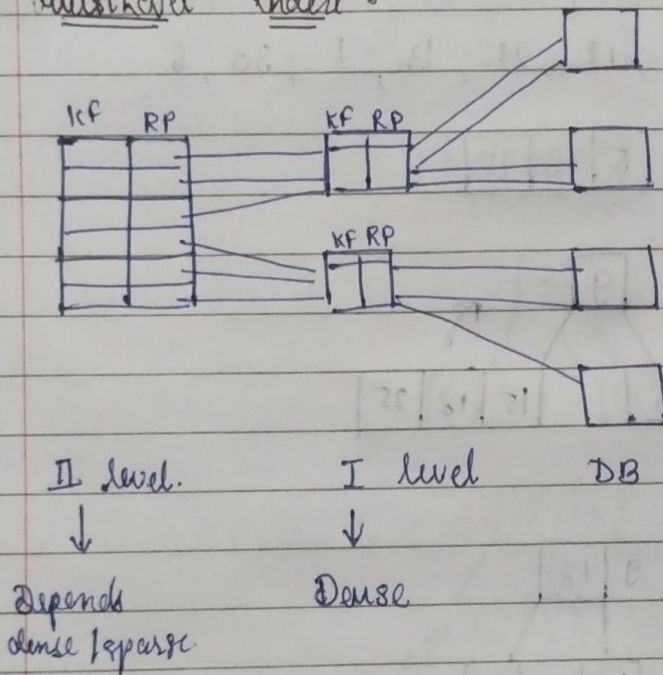
Sparse Index: per block only one entry.



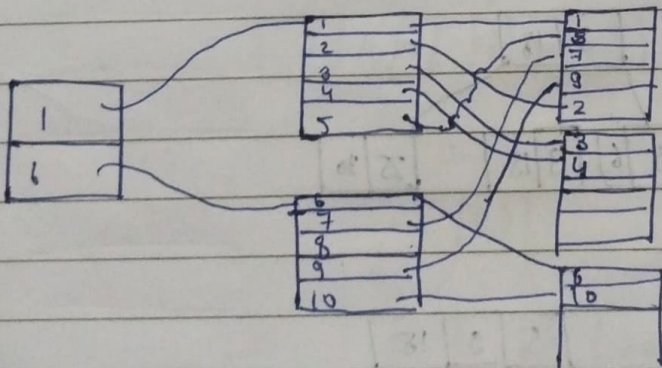
Problems in Indexing:

- large database more time
- growing of database dynamically.
- extra space for index. (log file is created)

Multi-level Index:



Index file size = block size.



B-Tree B-Tree is a multiway search tree in which per node we can have more than one key and within the node keys are sorted.

Order $\rightarrow n$

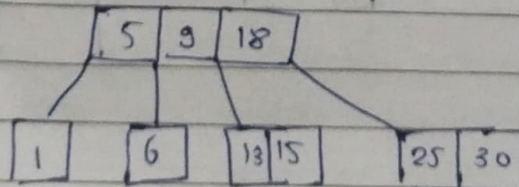
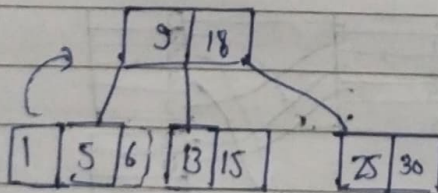
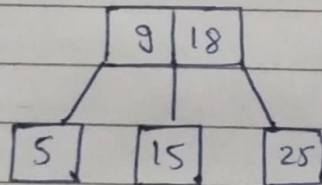
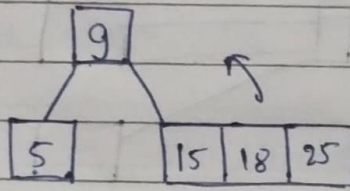
max $\rightarrow n-1$

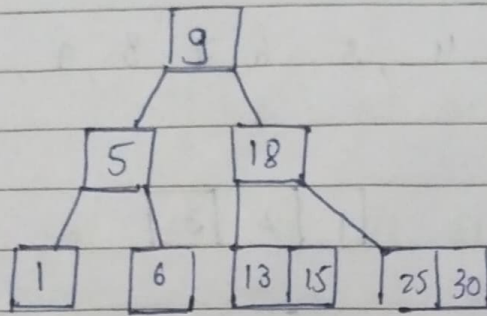
min $\rightarrow \lceil \frac{n}{2} \rceil - 1$

Ques

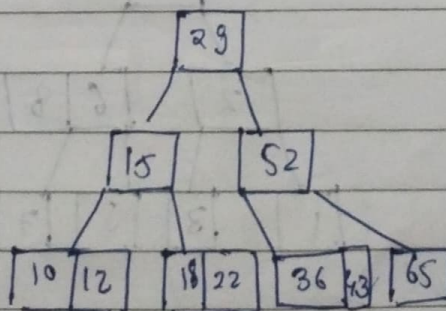
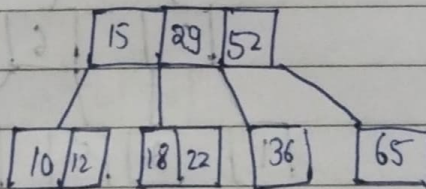
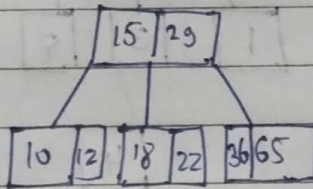
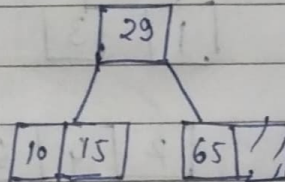
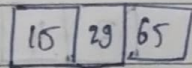
5, 9, 15, 18, 25, 13, 1, 30, 6

5 | 9 | 15

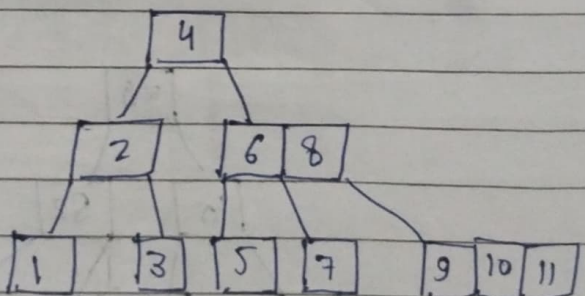
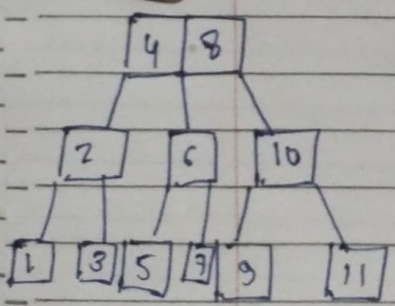
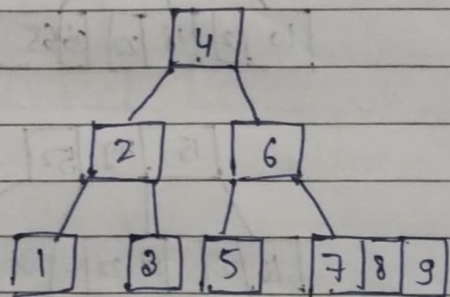
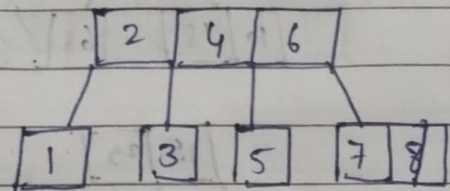
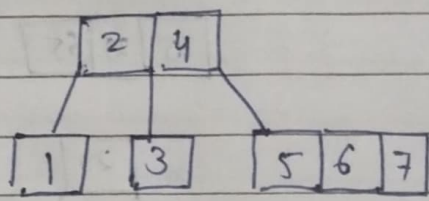
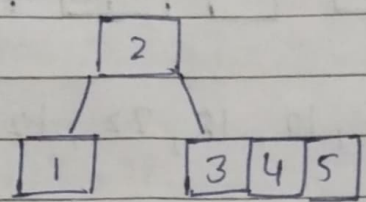
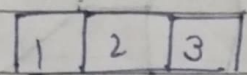




Ques 15, 29, 65, 10, 18, 22, 12, 36, 52, 43



Ans 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
make same
for order 4



B+ Tree

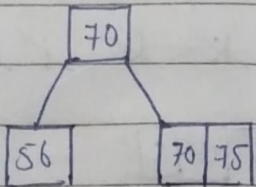
Order 3

{ we get one replication }

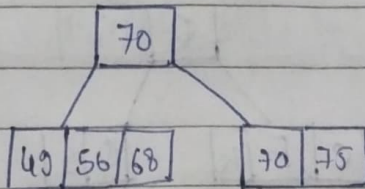
40, 75, 56, 68, 49, 42, 51, 76.

56 | 70 | 75

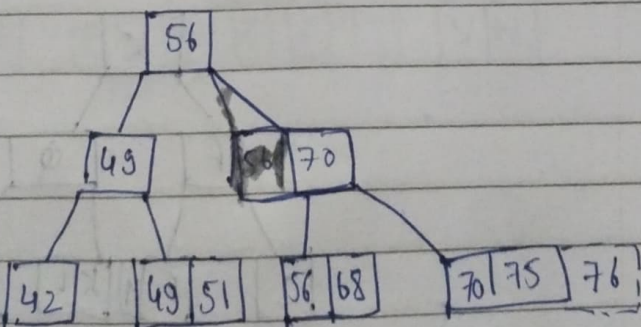
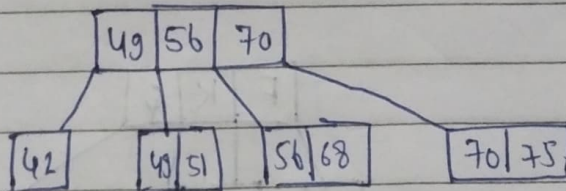
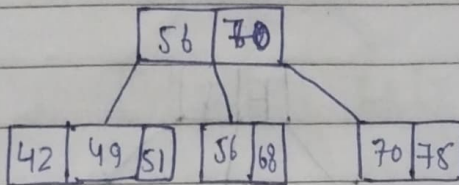
★ every element is sorted on leaf node.

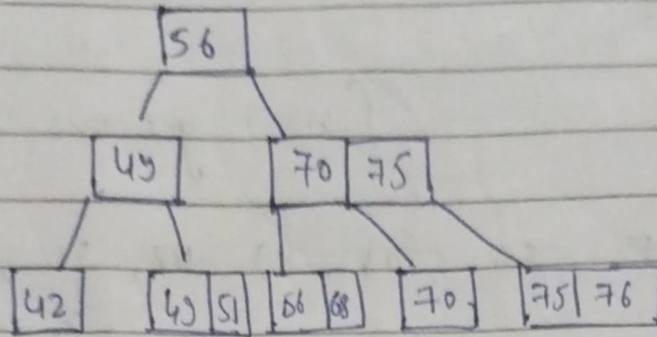


internally they are connected to linked list



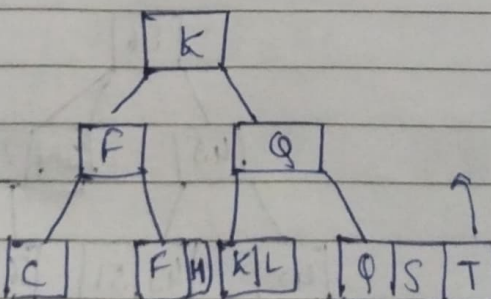
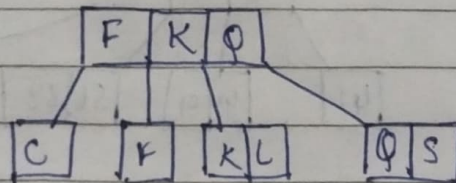
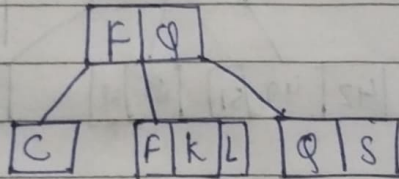
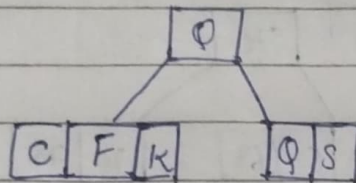
★ leaf node jab split hoga tab hi replication hoga warna nahi hoga

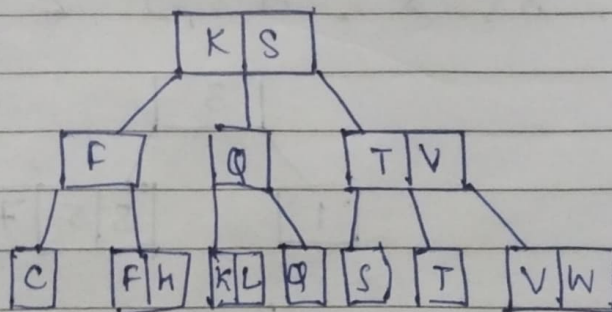
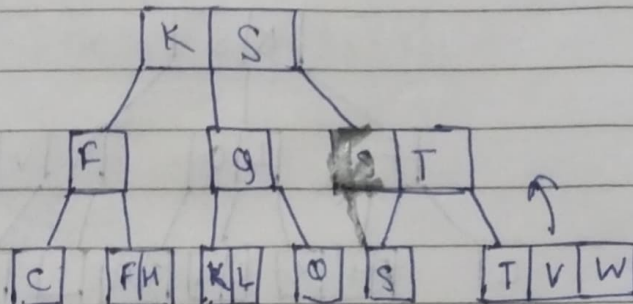
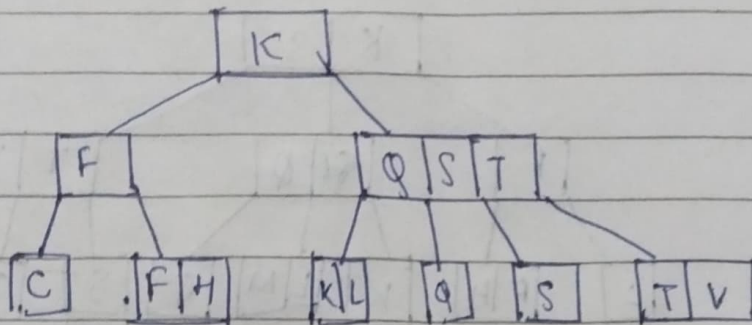
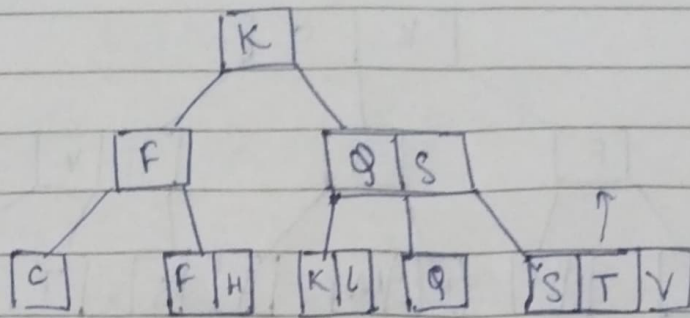


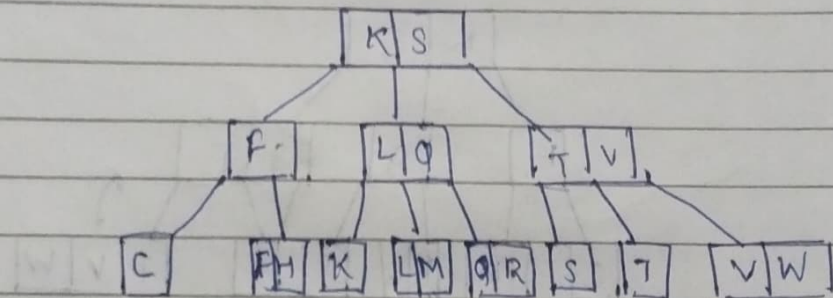
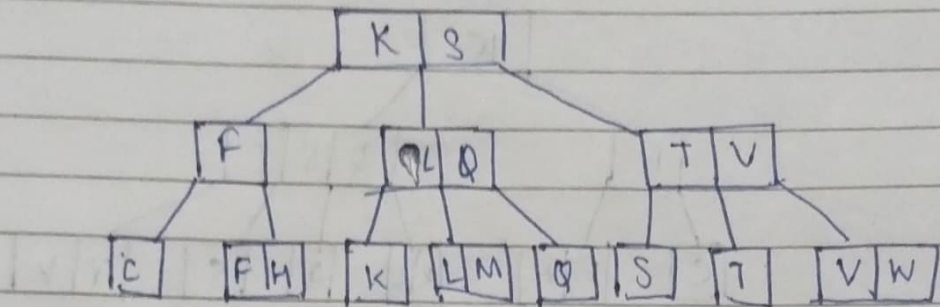
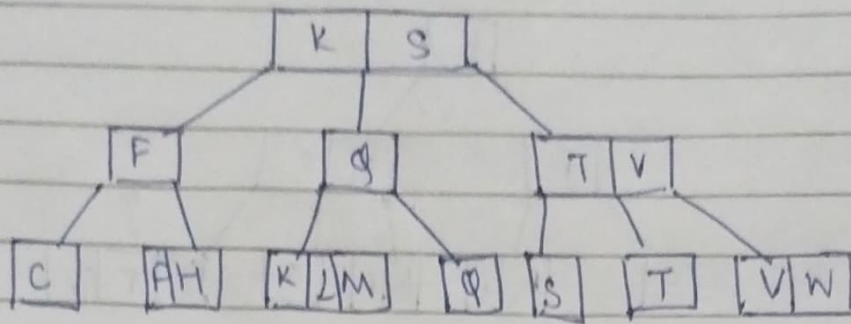


~~P~~ ~~S~~ ~~Q~~ ~~K~~ ~~G~~ ~~L~~ ~~H~~ ~~T~~ ~~V~~ ~~W~~ ~~M~~ ~~R~~

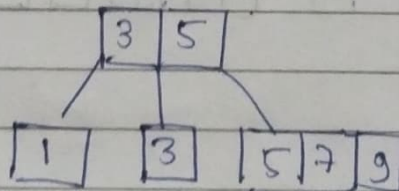
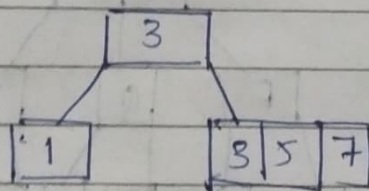
Order 3
F Q S

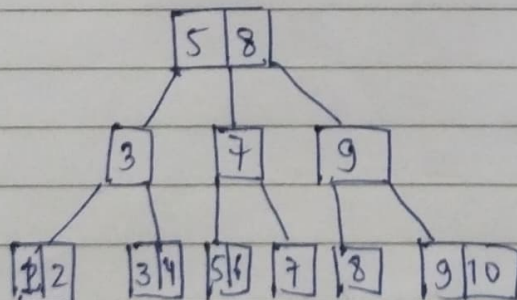
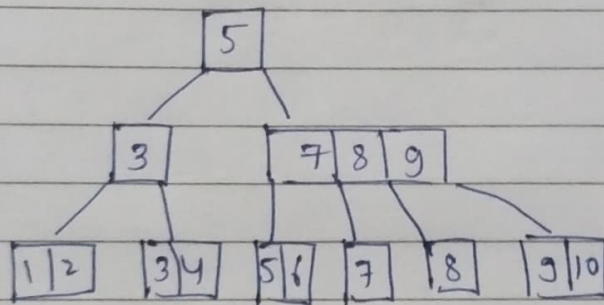
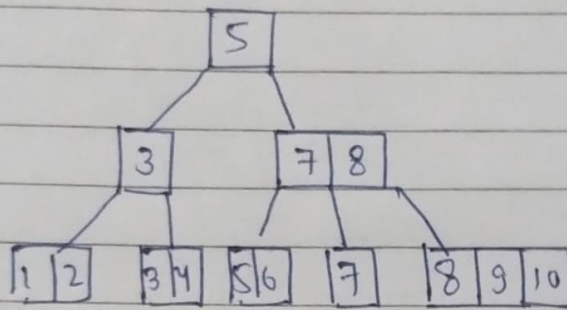
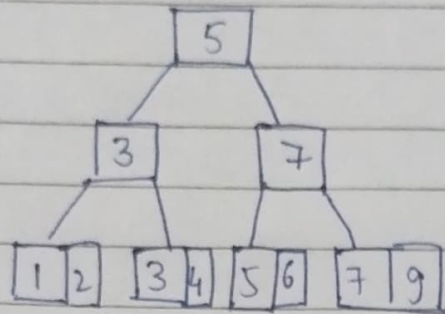
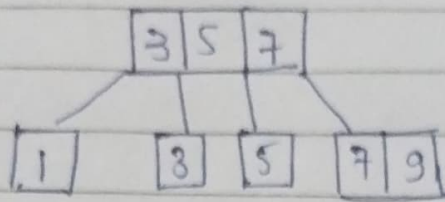






Ques 1, 3, 5, 7, 9, 2, 4, 6, 8, 10





Flashing

searching $\rightarrow$ find

0			
1	11		11, 55, 17, 9, 15, 2
2	2		$\downarrow$ $11 \div 10 = 1$ $55 \div 10 = 5$ $\downarrow$ remainder for all
3			
4			
5	55		
6			15 $\rightarrow$ 5 { already there }
7	17		
8			$\hookrightarrow$ collision: rehashing
9	9		

here searching will be O(1)
hashing technique

We make some hash function
for ex: $h(k) = k \bmod 10$

Two techniques for rehashing

If collision occurs

- $\rightarrow$ Rehashing (open addressing)
- $\rightarrow$ Chaining (close addressing)

Two techniques of rehashing

- $\rightarrow$ linear Probing
- $\rightarrow$ Quadratic Probing

Rehashing means change the hashing function.

Linear Probing :

If collision occurs we store the number in next empty block.

Ex. : 16, 10, 13, 11, 55, 65, 30, 36, 41, 62, 13

$$16 \% 7 = 2$$

$$10 \% 7 = 3$$

$$13 \% 7 = 6$$

$$11 \% 7 = 4$$

$$55 \% 7 = 6$$

$$65 \% 7 = 2$$

$$30 \% 7 = 2$$

$$36 \% 7 = 1$$

$$41 \% 7 = 6$$

$$62 \% 7 = 6$$

$$13 \% 7 = 6$$

$$0 \rightarrow 13$$

$$1 \rightarrow 36$$

$$2 \rightarrow 16$$

$$3 \rightarrow 10$$

$$4 \rightarrow 11$$

$$5 \rightarrow 65$$

$$6 \rightarrow 55$$

$$7 \rightarrow 30$$

$$8 \rightarrow 41$$

$$9 \rightarrow 62$$

Drawback

Primary

Clustering

Ques $h(x) = 3x \pmod{10}$ Indices {0, 1, ..., 9}

Keys : 1, 4, 5, 6, 14, 15

$$1 \rightarrow 3 \times 1 \pmod{10} = 3$$

$$15 \rightarrow 45 \pmod{10} = 5$$

$$4 \rightarrow 12 \pmod{10} = 2$$

$$5 \rightarrow 15 \pmod{10} = 5$$

$$6 \rightarrow 18 \pmod{10} = 8$$

$$14 \rightarrow 42 \pmod{10} = 2$$

0
 1
 2 $\rightarrow$ 4
 3 $\rightarrow$ 1
 4 $\rightarrow$ 14
 5 $\rightarrow$ 5
 6 $\rightarrow$ 15
 7
 8 $\rightarrow$ 6
 9

Index of
 14 and 15 are
 4 and 6.

Ques A hash table of length 10

$$h(k) = k \bmod 10$$

the table looks like ?

0 $\rightarrow$
 1 $\rightarrow$
 2 $\rightarrow$ 42
 3 $\rightarrow$ 23
 4 $\rightarrow$ 34
 5 $\rightarrow$ 52
 6 $\rightarrow$ 46
 7 $\rightarrow$ 33
 8 $\rightarrow$
 9 $\rightarrow$

which order in key values could have been inserted in table.

- 1) 46, 42, 34, 52, 23, 33
- 2) 34, 42, 23, 52, 33, 46
- 3) 46, 34, 42, 23, 52, 33
- 4) 42, 46, 33, 23, 34, 52

$$\begin{aligned} [27 \cdot 10] & \rightarrow 17 \cdot 10 = 1 \\ [34 \cdot 10] & \rightarrow 14 \cdot 10 = 1 \end{aligned}$$

$$\begin{aligned} i=2 & \rightarrow 47 \cdot 10 = 4 \\ & \quad 87 \cdot 10 = 8 \end{aligned}$$

$$\begin{aligned} i=9 & \rightarrow 81 \cdot 10 = 1 \\ & \quad 729 \cdot 10 = 9 \end{aligned}$$

If we see then $i^3 \cdot 10$ is distributing better than $i^2 \cdot 10$ no collision in this case

Chaining

$$h(k) = k \cdot 10$$

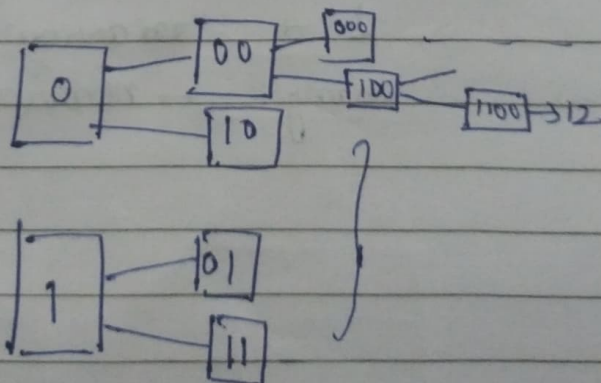
~~11, 22, 28, 27, 33, 43, 42, 51, 56~~

0	
1	11 → 51
2	22 → 42
3	23 → 33 → 43
4	
5	
6	56
7	27
8	
9	

Dynamic Hashing :

$$k \cdot 10 = 16$$

$$\begin{aligned} k=12 & : 12 \cdot 10 = 12 & 1100 \\ k=13 & : 13 \cdot 10 = 13 & 1101 \end{aligned}$$



Application

Hashing $\rightarrow$ select name from emp where empno = 10;

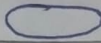
Indexing $\rightarrow$ select name from emp where empno between
10 to 50 ;
 Range

When exactly want to fetch one data
 $\rightarrow$ Hashing.

In case of range
 $\rightarrow$ Indexing (B+ Tree).

E-R Diagram

Entity $\rightarrow$ 

Object attribute $\rightarrow$ 

relation $\rightarrow$ 

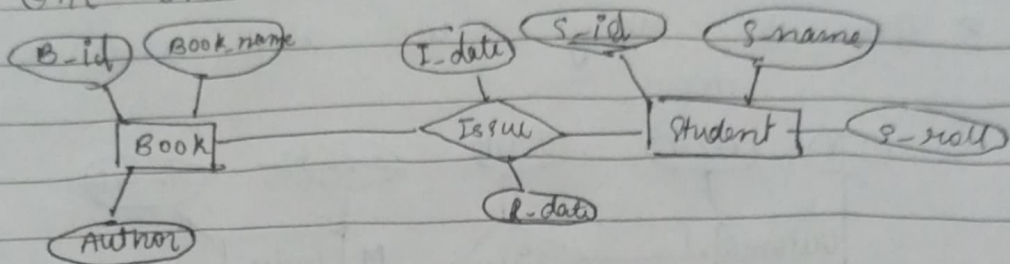
Relations :

- $\rightarrow$ 1-1 (One to One)
- $\rightarrow$ 1-N (One to Many)
- $\rightarrow$ M-N (Many to Many)

1 $\rightarrow$ ~~no~~ arrow

many $\rightarrow$ ~~no~~ arrow

Q One - One.



Book

B-id	B-name	Author

~~Issue~~

I-date	R-date

~~Student~~

Issue

B-id	S-id	I-date	R-date

Student

S-id	S name	S-roll

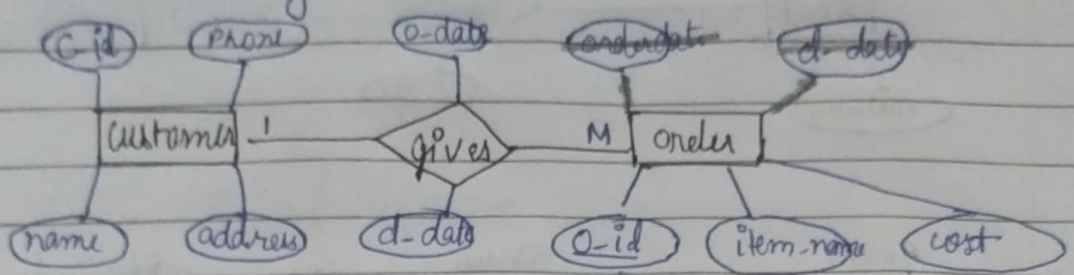
To reduce the number of table we can reduce only relation vala table.

→ If we make B-id as Primary key we can shift issue to Book (merge).

→ Or we make s-id as PK we can shift to student

So only 2 table will be enough.

One-to-Many : We shift table towards many.



customer

c-id	name	phone	address
c ₁			
c ₂			
c ₃			

gives

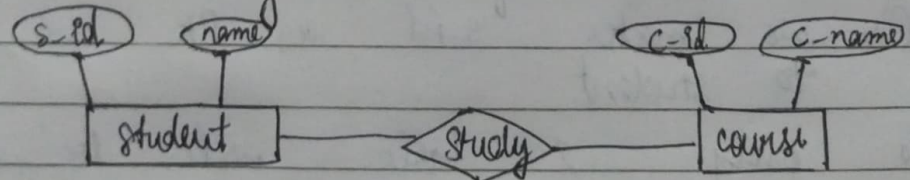
o-id	c-id	o-date	d-date

Order

o-id	item-name	cost

we shift gives towards order
Minimum table required : 02

Many to many



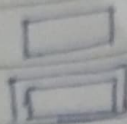
In this case
no merge.

s-id	name

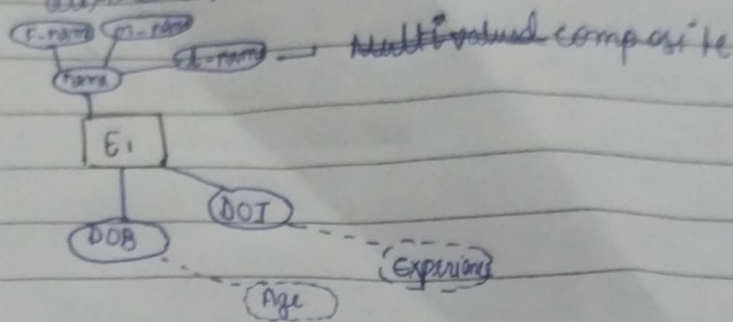
s-id	c-id
S ₁	C ₁
S ₁	C ₂
S ₂	C ₃
S ₁	C ₃

c-id	c-name

Strong entity
Weak entity.



If multivalued one more table is created for that attribute.



----- $\rightarrow$ derived.

composite $\rightarrow$ which can be broken
name can be broken into first name
middle name and last name.

single value attributes $\rightarrow$ A person can have
only one DOB

multi valued attributes $\rightarrow$ A person can have
multiple contacts

stored $\rightarrow$ DOB is stored

derived $\rightarrow$ Age is derived from DOB.

degree $\rightarrow$ no. of attributes

cardinality $\rightarrow$ no. of rows.

